

Principal Components Analysis

Last time

- Resampling Methods
- Machine Learning
- Qualitative Methods

This time

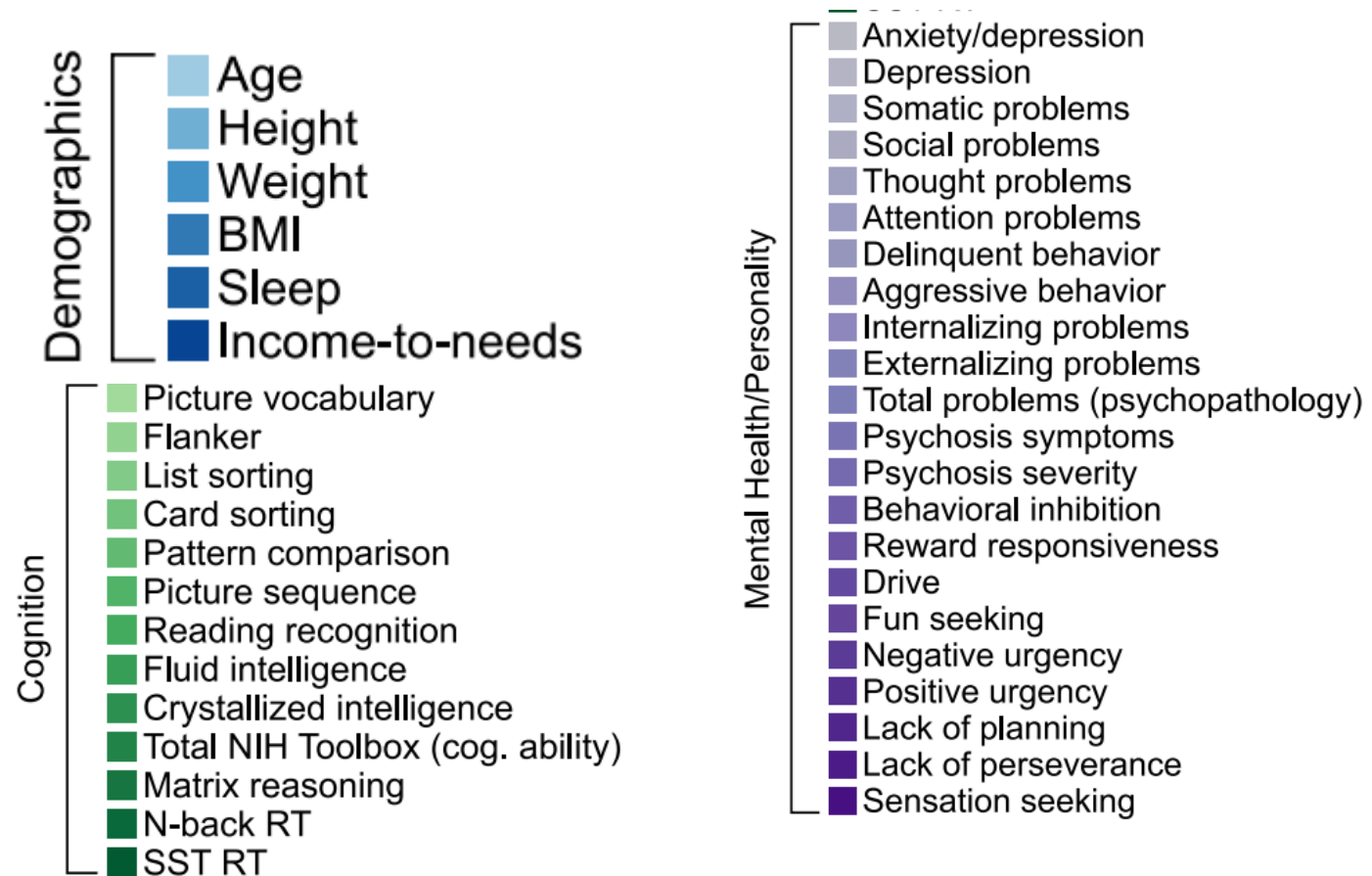
- Principal Components Analysis
- Much of this was taken from the **ISLR textbook**

Goals for today

- Why use PCA
- What is the general idea for PCA
- Compare/Contrast with Factor Analysis

The Problem

- You work with a lot of variables
- These variables are highly correlated
- What is this called?
- What's the problem with it?



ABCD study; Marek et al., 2022

If you wanted to visualize these 41 measures you'd need
820 bivariate scatterplots

PCA

What do we want?

- low-dimensional representation of the data...not 41 measures
- must capture as much information as possible!
- then we can use these reduced "components" for plotting or for subsequent analyses like regression!

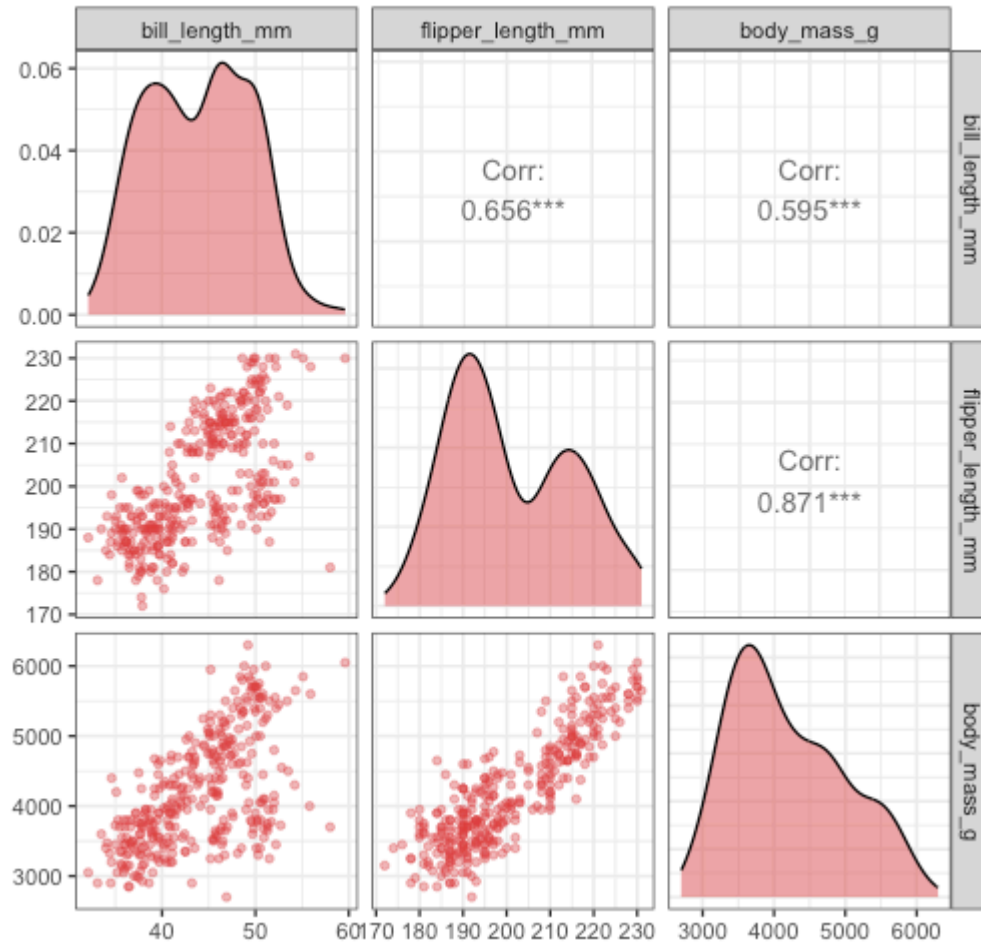
PCA

PCA is a method for re-expressing a set of variables

- Converting a large set of variables to a smaller number of **linear combinations that contain** *most* of the original information
- Exploring the dimensionality in the dataset
- Eliminating multicollinearity in a set of predictors

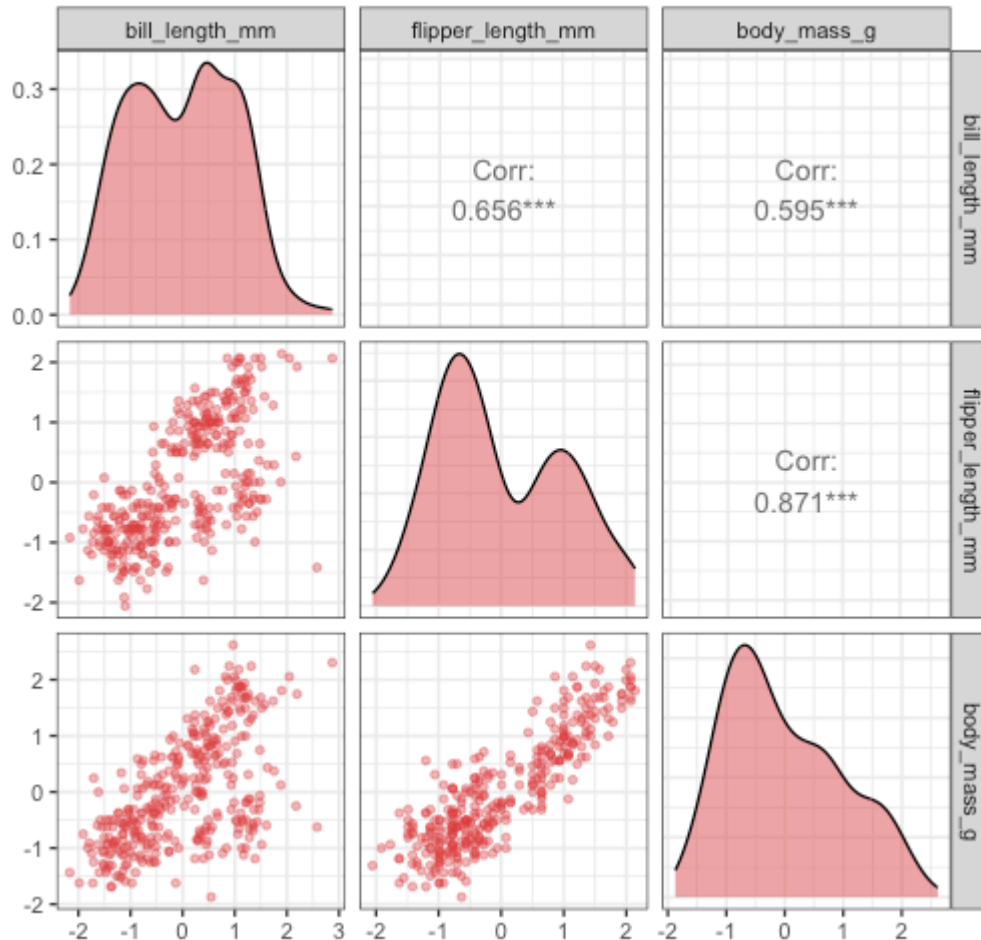
PCA Geometry/Visualized

UNSTANDARDIZED Bivariate relationships among penguin

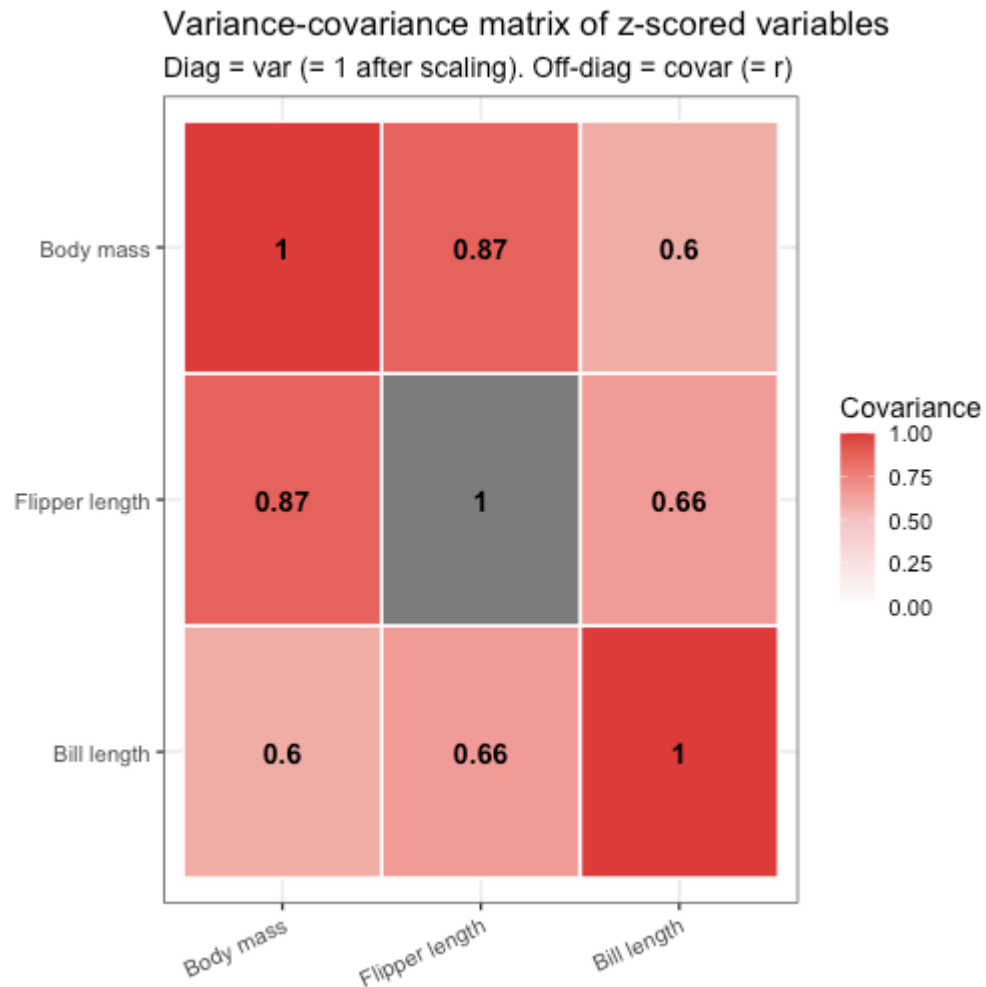


PCA Geometry/Visualized

Bivariate relationships after z-scoring



PCA Geometry/Visualized



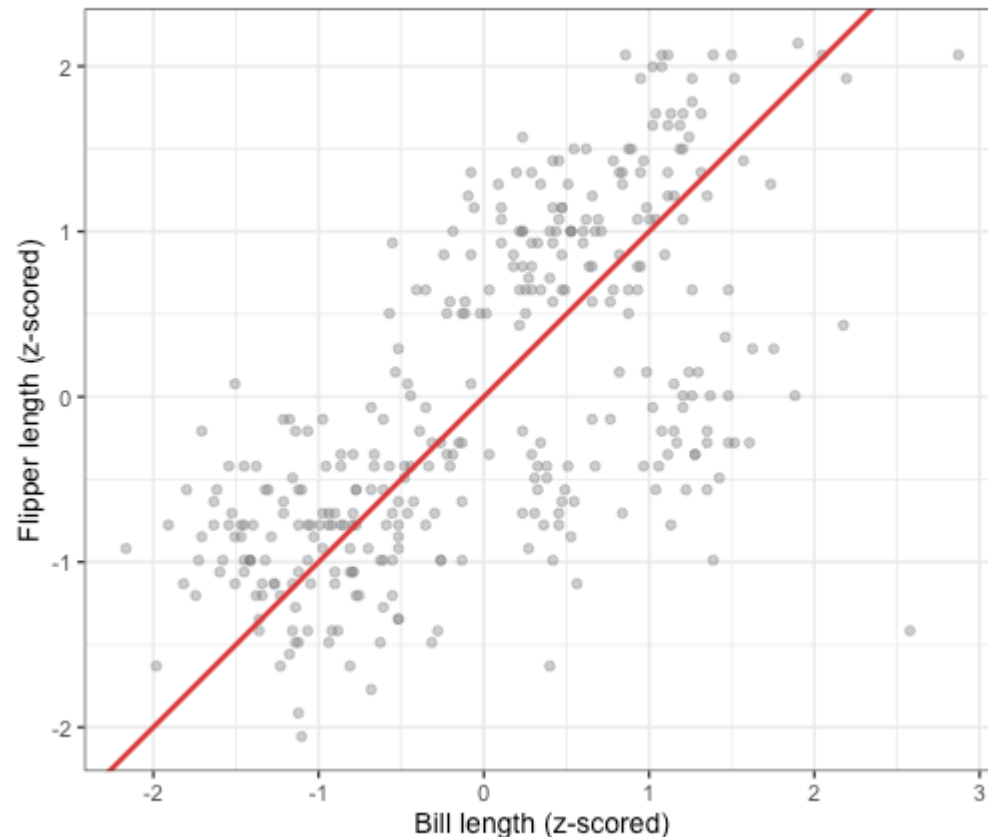
Now what? -- Eigendecomposition

What linear combination of these 3 variables captures the most variance?

1. Get a weighted sum of our 3 variables. Choose the weights to **maximize variance**
2. Do it again. Get a second weighted sum. But now, make sure that these weights are completely uncorrelated (**orthogonal**) to the first set.
 - The weights are our **loadings** (aka eigenvectors)
 - The **components** is the weighted sum of our original variables
 - How much of the total variance does each component capture? (eigenvalue)

PC1 as the axis of maximum variance

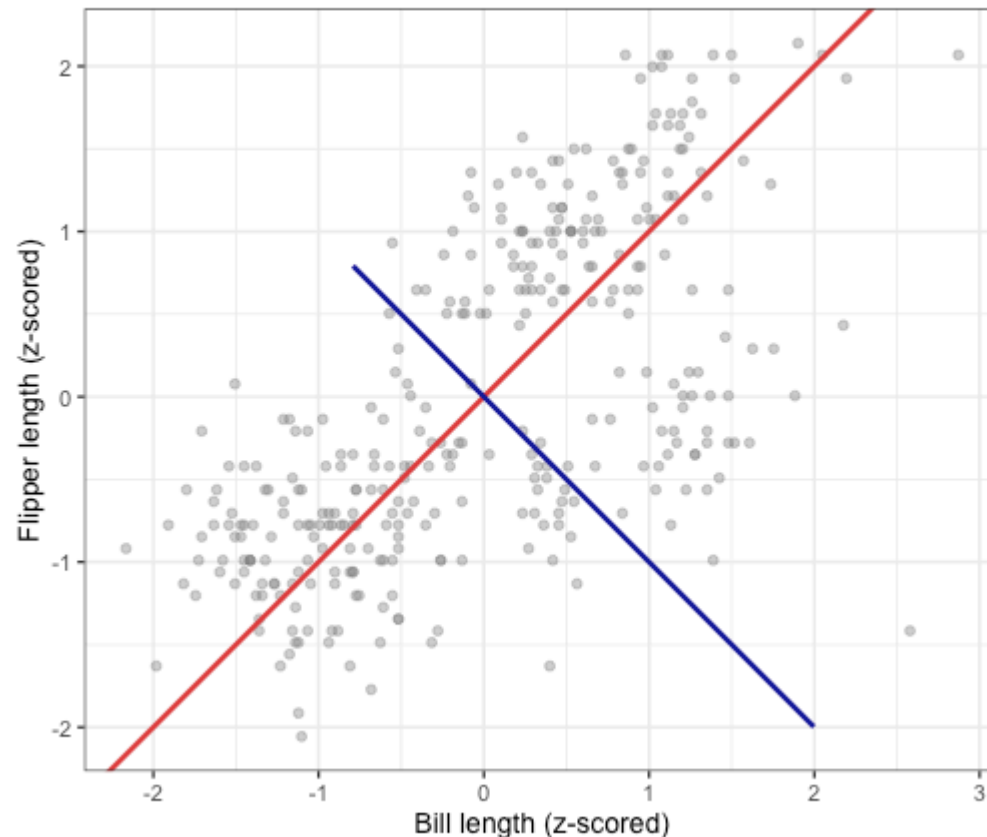
Red line = first principal component



Not quite OLS but looks similar! OLS uses vertical distance between points and line. PCA minimizes perpendicular distances between points and line -- no distinction between predictor and outcome

PC1 and PC2 as orthogonal axes through the data

Red = PC1 (max var) | Blue = PC2 (orthogonal, remaining var)



Orthogonality: Axes are 90deg angle so linearly uncorrelated. Each PC captures independent, non-overlapping variance; no more multicollinearity!

Interim Summary

What did we do?

- We took our standardized variables
- We found the axis that maximizes variance, and then the next orthogonal axis that maximizes variance (for however many variables you have)
- Started with 3 variables, but then only showed 2 so that I could do this with scatterplots (could have done with 3D plots but that's hard to read!)
- We did this via eigendecomposition (trust me). It gives us weights/loadings so that we can simply do this via linear combinations

Interim Summary

Why

- We have 41 variables, but only 4 principal components explain most of the variance!
- Transform our original data into linear combinations reflecting just those 4 components.
- No longer in z-score units. Data are projected into PC space. Interpretation of raw numbers is hard. However, the relationship is preserved. A penguin that was large in OG units is still large in PC space.
- BUT! I'm only down to 4 PCs. I no longer have 41 variables to work with, I'm down to 4. Goodbye multicollinearity!

Output

```
library(palmerpenguins)
```

```
penguins_sub <- penguins[, c("bill_length_mm", "flipper_length_mm", "body_mass_g")]  
penguins_sub <- na.omit(penguins_sub)  
penguins_scaled <- as.data.frame(scale(penguins_sub))
```

```
pca <- prcomp(penguins_scaled)  
pca
```

```
## Standard deviations (1, .., p=3):
```

```
## [1] 1.5562903 0.6734433 0.3527532
```

```
##
```

```
## Rotation (n x k) = (3 x 3):
```

```
##
```

	PC1	PC2	PC3
--	-----	-----	-----

## bill_length_mm	0.5284815	0.8429870	0.1004004
-------------------	-----------	-----------	-----------

## flipper_length_mm	0.6072904	-0.2927549	-0.7385750
----------------------	-----------	------------	------------

## body_mass_g	0.5932164	-0.4512954	0.6666534
----------------	-----------	------------	-----------

Output

```
head(pca$x)
```

##		PC1	PC2	PC3
##	[1,]	-1.661014	-0.07568742	0.58181156
##	[2,]	-1.369371	-0.14615908	0.36811264
##	[3,]	-1.310087	0.09950074	-0.54709716
##	[4,]	-1.596995	-0.52726754	-0.34199547
##	[5,]	-1.326938	-0.17590557	0.02964652
##	[6,]	-1.772838	-0.03622537	0.47422288

PC Scores -- linear combinations

$$PC1_i = w_1 \cdot billLength_i + w_2 \cdot flipperLength_i + w_3 \cdot bodyMass_i$$

- Take penguin 1's score
- Multiply it by weight

```
penguin_1 <- as.numeric(penguins_scaled[1, ])  
loadings_pc1 <- pca$rotation[, 1]  
  
manual_score <- sum(penguin_1 * loadings_pc1)  
manual_score
```

```
## [1] -1.661014
```

```
# verify it matches prcomp output  
pca$x[1, 1]
```

```
##          PC1  
## -1.661014
```

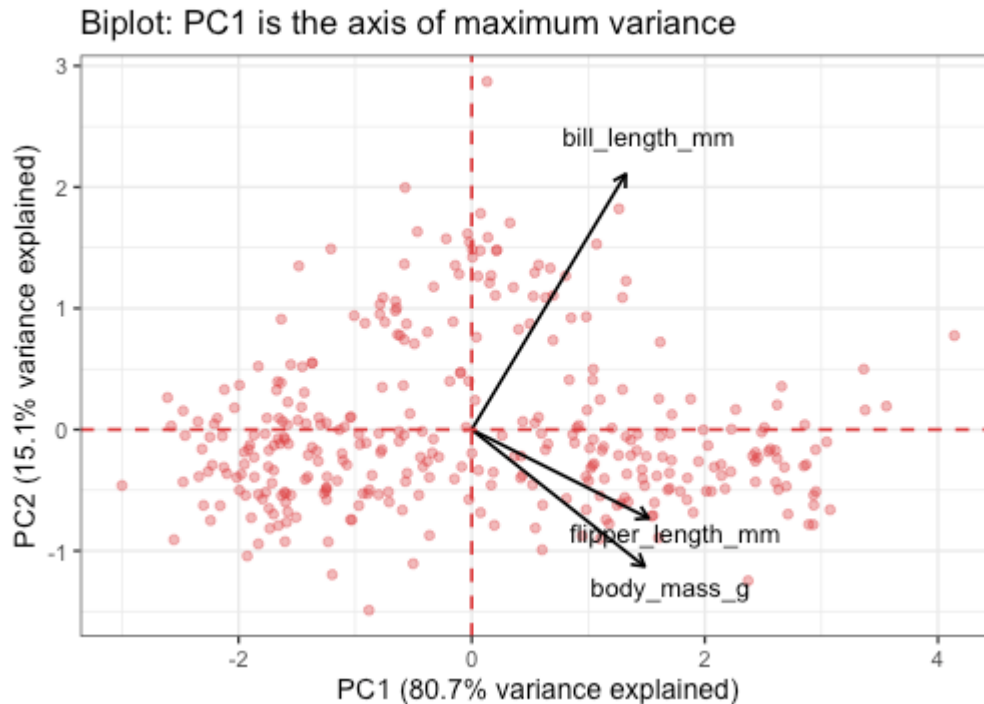
PC Scores -- linear combinations

$$PC1_i = w_1 \cdot billLength_i + w_2 \cdot flipperLength_i + w_3 \cdot bodyMass_i$$

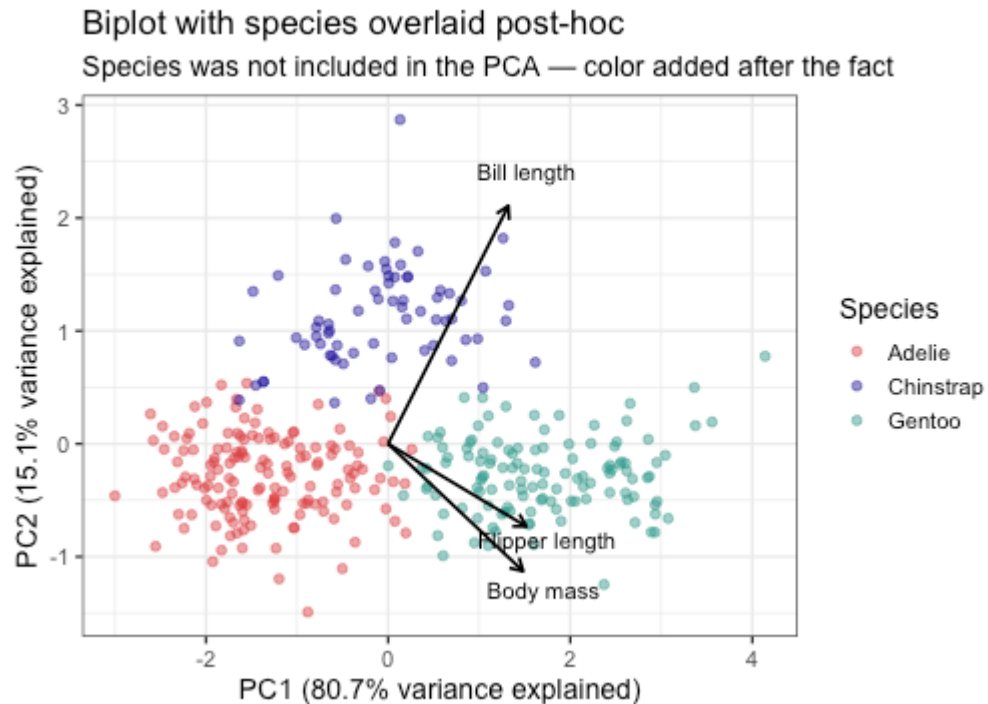
Multiply each z-scored variable by its corresponding loading, then sum all 3 products. This is the number where that penguin (observation) sits along the PC1 axis.

Do it again with the PC2 loadings and that's where it sits along the PC2 axis.

We like to project/plot these against each other -- **biplot**



Each point is a penguin in PC space. If PC1 reflects "body size" then a point on the right is a penguin that is large in all 3 variables. PC2 is secondary structure, after PC1 is accounted for. Arrows are an original variable. Straight up is entirely PC1 and no PC2. Diagonal is both PC1 and PC2. Small angle between them means pos correlation. 90 is no correlation. 180 is negative correlation.



Species was not in the PCA. But the variance structure is such that Gentoo penguins are strongly along PC1 (horizontal axis). *Body size variables alone were strong enough to implicitly distinguish species.* You can recover groups you didn't explicitly model! BUT! You can always color code by something after the fact. Psych data isn't as clean. PCA can detect variance structures.

It cannot tell you what that structure means or what label to assign the component -- that is your job!

Dimensionality Reduction

- Supposedly used for dimensionality reduction
- But we still have 3 PCs

```
## Importance of components:
##               PC1      PC2      PC3
## Standard deviation      1.5563 0.6734 0.35275
## Proportion of Variance 0.8074 0.1512 0.04148
## Cumulative Proportion 0.8074 0.9585 1.00000

## Standard deviations (1, ..., p=3):
## [1] 1.5562903 0.6734433 0.3527532
##
## Rotation (n x k) = (3 x 3):
##               PC1      PC2      PC3
## bill_length_mm      0.5284815 0.8429870 0.1004004
## flipper_length_mm 0.6072904 -0.2927549 -0.7385750
## body_mass_g         0.5932164 -0.4512954 0.6666534
```

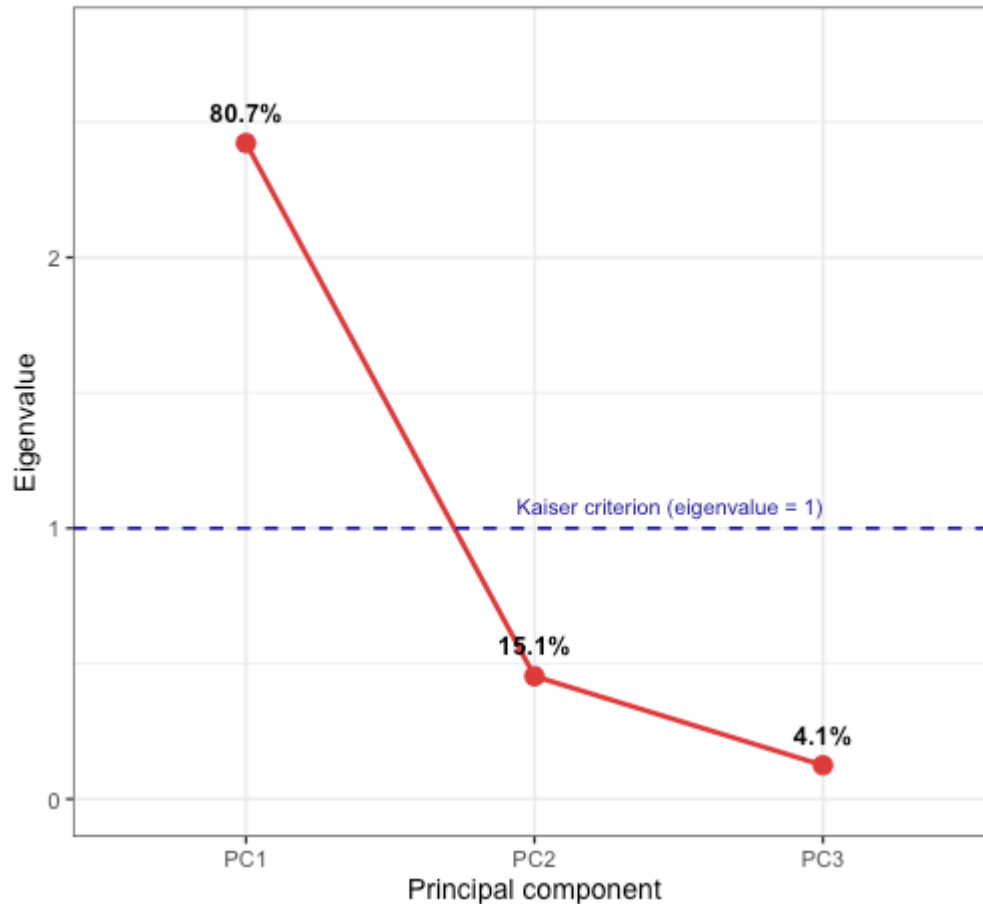
You get to choose what to keep!

Which PCs are worth it?

- PCA simply rotates the coordinate system
- Dimensionality reduction is a subsequent **decision** you make, not something PCA does automatically
- Keep the ones that explain a lot of variance, and get rid of ones that don't

Scree plot

Eigenvalue = variance captured by each component



- Keep PCs with eigenvalue > 1
- Keep above the elbow
- Parallel analysis (best practice)

The Payoff

```

penguins_model <- penguins[!is.na(penguins$bill_length_mm) &
                           !is.na(penguins$flipper_length_mm) &
                           !is.na(penguins$body_mass_g), ]

penguins_model$PC1 <- pca$x[, 1]

# example: predict bill depth from PC1
fit <- lm(bill_depth_mm ~ PC1, data = penguins_model)
summary(fit)

```

```

##
## Call:
## lm(formula = bill_depth_mm ~ PC1, data = penguins_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5472 -1.2452 -0.0185  1.0751  4.5490
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  17.15117    0.09337   183.7  <2e-16 ***
## PC1          -0.61863    0.06008   -10.3  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.727 on 340 degrees of freedom
## Multiple R-squared:  0.2377,    Adjusted R-squared:  0.2354

```

```
##
## Call:
## lm(formula = bill_depth_mm ~ PC1, data = penguins_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5472 -1.2452 -0.0185  1.0751  4.5490
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  17.15117    0.09337   183.7  <2e-16 ***
## PC1          -0.61863    0.06008   -10.3  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.727 on 340 degrees of freedom
## Multiple R-squared:  0.2377,    Adjusted R-squared:  0.2354
## F-statistic:   106 on 1 and 340 DF,  p-value: < 2.2e-16
```

Note 1: Even if we kept multiple PCs and used them as variables, they are perfectly uncorrelated with each other — orthogonality is guaranteed. This is one of the cleanest solutions to multicollinearity available.

```
##
## Call:
## lm(formula = bill_depth_mm ~ PC1, data = penguins_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5472 -1.2452 -0.0185  1.0751  4.5490
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  17.15117    0.09337   183.7  <2e-16 ***
## PC1          -0.61863    0.06008   -10.3  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.727 on 340 degrees of freedom
## Multiple R-squared:  0.2377,    Adjusted R-squared:  0.2354
## F-statistic:   106 on 1 and 340 DF,  p-value: < 2.2e-16
```

Note 2: "A 1-unit increase along the PC1 direction predicts a .61 decrease in bill depth". Here this might be interpretable. What if it's those 41 variables and you keep 4 components but it's not cleanly a "mental health" or "SES" component?

```
##
## Call:
## lm(formula = bill_depth_mm ~ PC1, data = penguins_model)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -3.5472 -1.2452 -0.0185  1.0751  4.5490
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  17.15117    0.09337   183.7  <2e-16 ***
## PC1          -0.61863    0.06008   -10.3  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.727 on 340 degrees of freedom
## Multiple R-squared:  0.2377,    Adjusted R-squared:  0.2354
## F-statistic:   106 on 1 and 340 DF,  p-value: < 2.2e-16
```

Note 3: When you discard a PC, that is an implicit assumption. You are saying that those are not relevant to your outcome. If PC2 does actually predict your outcome, and you threw it away, now you have a model specification error.

What does PCA solve?

- Reduces dimensionality while retaining as much variance as possible
- Solves multicollinearity -- PCs are perfectly orthogonal
- You can use this on highly dimensional datasets where visualizing/modeling is not feasible
- The scores are just new variables — you can use them how you see fit
- PCA is a data exploration step -- not a preprocessing trick. It forces you to think about the nature of your data. Which variables should covary? Are there surprises?

What does PCA *not* solve?

- Nature does not owe you simplicity
- Interpretation is your responsibility/job, not the algorithm's
- When you have a regression, you know Y. You know the answer. You can check your work.
- Here, there's no right answer, no goal necessarily. There is no Y!!!

there is no way to check our work because we don't know the true answer—the problem is **unsupervised** -- ISLR

PCA vs. Confirmatory Factor Analysis

	PCA	CFA
Goal	Summarize variance in observed variables	Test whether latent factors explain observed variables
Direction	Data-driven	Theory-driven
Latent vars?	No — components are weighted sums, not causes	Yes — factors are hypothesized to <i>cause</i> the items
Evaluation	No fit index — no wrong answer	Model fit is testable (CFI, RMSEA, etc.)
Best for	Exploration, dimensionality reduction	Confirming a theorized factor structure

PCA vs. Exploratory Factor Analysis

- Technically, these are different
- Practically, they aren't different enough to worry about

Other dimensionality reduction methods

- **Exploratory Factor Analysis (EFA)** — like PCA but models only shared variance, assuming latent factors cause item covariance
- **Independent Component Analysis (ICA)** — finds statistically independent (not just uncorrelated) components; ubiquitous in EEG/fMRI artifact separation
- **Multidimensional Scaling (MDS)** — reduces dimensions while preserving pairwise distances between observations; common in similarity/dissimilarity judgment tasks
- **Canonical Correlation Analysis (CCA)** — finds linear combinations that maximize correlation between two sets of variables simultaneously; used when you have two multivariate batteries

Why cover this in Quant II?

- It's really useful in Psychology & Neuroscience research -
- you will see PCA quite often
- High-dimensional data often has hidden low-dimensional structure, and that we can find it without knowing the answer in advance — no Y required.
- This idea doesn't stop at PCA. What happens when you scale that intuition up by orders of magnitude — more data, more dimensions, and methods that can capture nonlinear structure PCA can't touch? -- **Artificial Intelligence**
- The core question is the same: what is the underlying structure of complex data, and how do we find it?

Next time

An Introduction to AI with Dr. Oltmanns